扫码可进资料分享群

使用Makefile、VCS、Verdi 做个简单的 Test Bench

目录：

1. 简介
2. 需求
3. 加法器模块
4. 测试模块
5. 测试脚本
6. 编译项目
7. 测试结果

扫码可进资料分享群

1. 简介

Synopsys 的 VCS 和 Verdi 是做 IC 使用的很好的开发工具。但新手往往是无法下手，入门比较困难。在此，我根据自己的学习经历，写个最简单的使用教程。教程中会用到 Makefile、VCS、Verdi，写个简单的 8 位加法器的 TB 例子。所有代码都使用 verilog 编写，带简单的结果验证功能。

此教程没有使用到 UVM，以后有时间我在单独写个 UVM 的简单例子。

扫码可进资料分享群

2. 需求

我是在 VMware 下开发测试的，用到的软件列表如下：

- VMware® Workstation 12 Pro，12.5.7 build-5813279
- CentOS-6.10-x86_64-bin-DVD1.iso
- scl_v2016.12_common.spf
- scl_v2016.12_linux64.spf
- SynopsysInstaller_v3.5.run
- uvm-1.1d.tar.gz
- VCS_vL-2016-SP2-12
- verdi_vL-2016.06-SP2-12

Win7 下的许可证破解软件是：

- scl_keygen.rar

扫码可进资料分享群

3. 加法器模块

8 位加法器 adder8.v 代码如下：

```
module adder8 (
    input clk,
    input [7:0] a_i,
    input [7:0] b_i,
    output reg [8:0] c_o
);

always @ (posedge clk) begin
    c_o <= a_i + b_i;
end

endmodule
```

输入 a_i 和 b_i 都是 8 位的，输出 c_o 是 9 位的。只是用做示例，不需要太纠结合理性了。

扫码可进资料分享群

4. 测试模块

测试模块 tb_adder8.v，代码如下：

```
// TB_SEED 是随机种子
`ifndef TB_SEED
`define TB_SEED 0
`endif

module tb_adder8 ();
   wire [8:0] result;
   reg [7:0] input_0;
   reg [7:0] input_1;
   reg clk;

   // clk2 是主 clk 的延迟，用于验证结果
   wire #5 clk2;
   assign  clk2 = clk;

initial begin
   $fsdbDumpfile("adder8.fsdb");
   $fsdbDumpvars();

   $display("TB_SEED is %d", `TB_SEED);

   clk = 0;
   input_0 = 8'd0;
   input_1 = 8'd0;

   #10000
   $display("All test PASS!");
   $finish;
end

// 主时钟 50MHz
always begin
   #10 clk = ~clk;
end

// 产生随机输入
always @ (negedge clk) begin
   input_0 = $random() % 256;
   input_1 = $random() % 256;
end

// 获取验证输出
always @ (posedge clk2) begin
   if ((input_0 + input_1) != result) begin
      $display("Test failed for %x + %x = %x", input_0, input_1, result);
      $finish;
   end else begin
      $display("%x + %x = %x", input_0, input_1, result);
```

扫码可进资料分享群

```
    end
end

// 连接加法器模块
adder8 dut(
    .clk(clk),
    .a_i(input_0),
    .b_i(input_1),
    .c_o(result)
 );

endmodule
```

扫码可进资料分享群

5. 测试脚本

整个编译过程采用 Makefile 控制，Makefile 文件内容如下：

```
VCS = vcs -sverilog -timescale=1ns/1ns +vpi -l build.log -debug_access+all
SIMV =  ./simv  -l simv.log

ifndef TB_SEED
TB_SEED = 1024
endif

all: comp run

comp:
    $(VCS) +define+TB_SEED=$(TB_SEED) +incdir+. \
        adder8.v \
        tb_adder8.v

run:
    $(SIMV) +fsdbfile+top.fsdb

dbg:
    verdi -f file.f -ssf top.fsdb &

clean:
    rm -rf core csrc simv* vc_hdrs.h ucli.key urg* *.log *.fsdb novas.* verdiLog
```

Makefile 里面的空格排版都是 TAB 键，否则会出错，修改的时候请注意一下这个细节。

其中 **file.f** 文件的内容如下：

```
tb_adder8.v
adder8.v
```

扫码可进资料分享群

6. 编译项目

编译前，请先清除项目：

```
make clean
```

然后，编译项目：

```
make
```

然后，查看波形：

```
make dbg
```

扫码可进资料分享群

编译日志文件为：build.log，如果有错误可查看该文件：

```
Command: vcs -sverilog -timescale=1ns/1ns +vpi -l build.log -debug_access+all
+define+TB_SEED=1024 \
+incdir+. adder8.v tb_adder8.v
                         Chronologic VCS (TM)
     Version L-2016.06-SP2-12_Full64 -- Tue Nov 20 18:57:09 2018
               Copyright (c) 1991-2016 by Synopsys Inc.
                         ALL RIGHTS RESERVED

This program is proprietary and confidential information of Synopsys Inc.
and may be used and disclosed only as authorized in a license agreement
controlling such use and disclosure.

Parsing design file 'adder8.v'
Parsing design file 'tb_adder8.v'
Top Level Modules:
       tb_adder8
TimeScale is 1 ns / 1 ns
Starting vcs inline pass...
1 unique modules to generate
recompiling module tb_adder8
make[1]: Entering directory `/home/toor/ajob/adder8/csrc'
rm -f _csrc*.so pre_vcsobj_*.so share_vcsobj_*.so
if [ -x ../simv ]; then chmod -x ../simv; fi
g++  -o ../simv   -Wl,-rpath-link=./ -Wl,-rpath='$ORIGIN'/simv.daidir/ -Wl,-
rpath=./simv.daidir/ \
-Wl,-rpath='$ORIGIN'/simv.daidir//scsim.db.dir  -rdynamic   objs/amcQw_d.o
_19474_archive_1.so \
SIM_l.o       rmapats_mop.o rmapats.o rmar.o  rmar_llvm_0_1.o rmar_llvm_0_0.o    \
/usr/synopsys/vcs_L-2016.06-SP2-12/linux64/lib/libzerosoft_rt_stubs.so
/usr/synopsys/vcs_L-2016.06-SP2-12/linux64/lib/libvirsim.so \
/usr/synopsys/vcs_L-2016.06-SP2-12/linux64/lib/liberrorinf.so /usr/synopsys/vcs_L-
2016.06-SP2-12/linux64/lib/libsnpsmalloc.so \
/usr/synopsys/vcs_L-2016.06-SP2-12/linux64/lib/libvfs.so    /usr/synopsys/vcs_L-
2016.06-SP2-12/linux64/lib/libvcsnew.so \
/usr/synopsys/vcs_L-2016.06-SP2-12/linux64/lib/libsimprofile.so /usr/synopsys/vcs_L-
2016.06-SP2-12/linux64/lib/libuclinative.so \
-Wl,-whole-archive /usr/synopsys/vcs_L-2016.06-SP2-12/linux64/lib/libvcsucli.so -Wl,-
no-whole-archive \
_vcs_pli_stub_.o   /usr/synopsys/vcs_L-2016.06-SP2-
12/linux64/lib/vcs_save_restore_new.o \
/usr/synopsys/verdi3_L-2016.06-SP2-12/share/PLI/VCS/LINUX64/pli.a -ldl  -lc -lm -
lpthread \
-ldl
../simv up to date
make[1]: Leaving directory `/home/toor/ajob/adder8/csrc'
CPU time: .294 seconds to compile + .427 seconds to elab + .276 seconds to link
```

扫码可进资料分享群

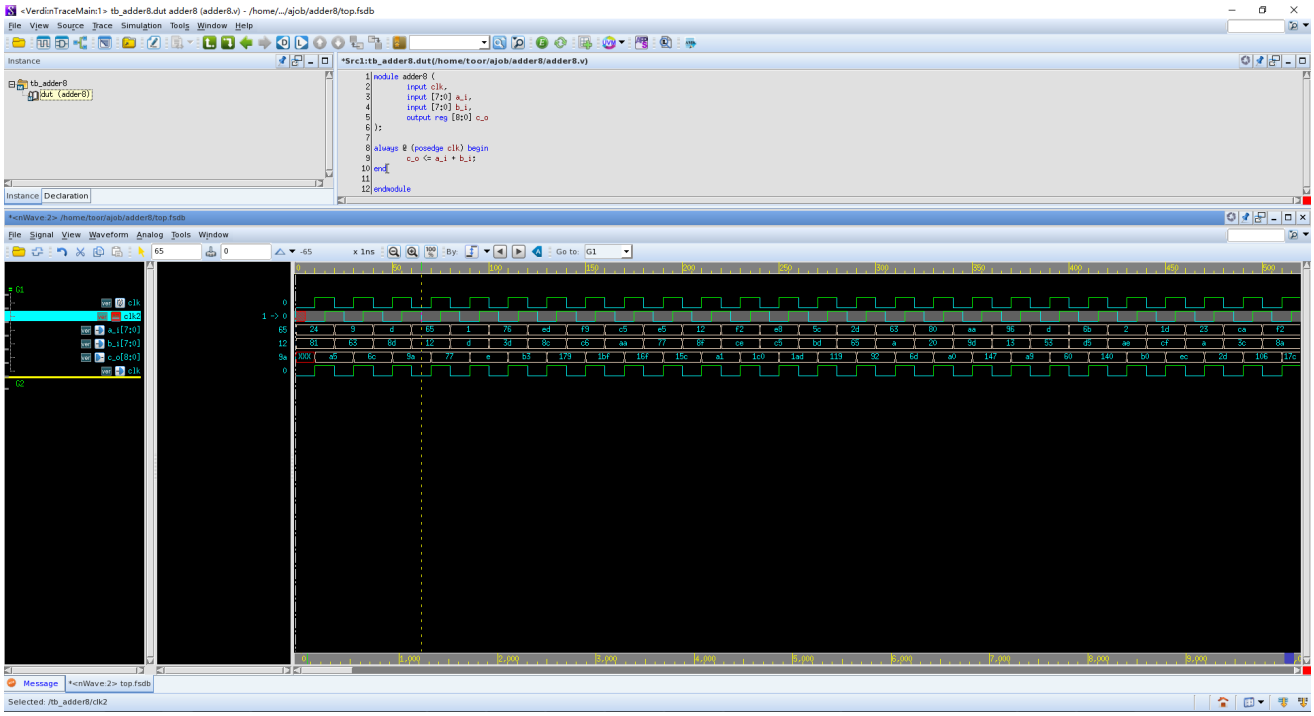
仿真日志文件为：simv.log，仿真结果在该文件内:

```
Command: /home/toor/ajob/adder8/./simv -l simv.log +fsdbfile+top.fsdb
Chronologic VCS simulator copyright 1991-2016
Contains Synopsys proprietary information.
Compiler version L-2016.06-SP2-12_Full64; Runtime version L-2016.06-SP2-12_Full64;  Nov
20 18:57 2018
*Verdi3* Loading libsscore_vcs201606.so
*Verdi3* : FSDB_GATE is set.
*Verdi3* : FSDB_RTL is set.
*Verdi3* : Enable Parallel Dumping.
FSDB Dumper for VCS, Release Verdi3_L-2016.06-SP2-12, Linux x86_64/64bit, 12/11/2017
(C) 1996 - 2017 by Synopsys, Inc.
*Verdi3* : Create FSDB file 'top.fsdb'
*Verdi3* : Begin traversing the scopes, layer (0).
*Verdi3* : End of traversing.
TB_SEED is        1024
24 + 81 = 0a5
09 + 63 = 06c
0d + 8d = 09a
65 + 12 = 077
01 + 0d = 00e
76 + 3d = 0b3
ed + 8c = 179
f9 + c6 = 1bf
c5 + aa = 16f
...
...
...
51 + 04 = 055
80 + f9 = 179
06 + ca = 0d0
All test PASS!
$finish called from file "tb_adder8.v", line 26.
$finish at simulation time                10000
           V C S   S i m u l a t i o n   R e p o r t
Time: 10000 ns
CPU Time:      0.550 seconds;       Data structure size:   0.0Mb
Tue Nov 20 18:57:11 2018
```

扫码可进资料分享群

7. 测试结果

最终 Verdi 的效果如下：



这样一个简单的采用 Makefile、VCS、Verdi 的 Test Bench 就诞生了。